

## Lezione 2 – HTTP

Reti di calcolatori

Modulo 3 - Protocolli applicativi

Unità didattica 4 – Protocolli del web

**Ernesto Damiani**

---

Università degli Studi di Milano - Ssri - CDL ONLINE

## HTTP

### HTTP (HyperText Transfer Protocol)

- Linguaggio del Web
  - Protocollo usato per la comunicazione tra i browser web e i server web
- Porta TCP 80
- RFC 1945

## URI, URN, URL

- **URI (Uniform Resource Identifier)**
  - Informazioni relative a una risorsa
- **URN (Universal Resource Name)**
  - Il nome della risorsa nel namespace
- **URL (Uniform Resource Locator)**
  - Come individuare la risorsa
  - Un URI che dice come trovare la risorsa

## URL

- URL
  - Protocollo (http, ftp, news)
  - Nome dell'host (nome.dominio)
  - Porta (di solito 80, ma molti Web server rispondono sulla 8080)
  - Percorso di filesystem alla risorsa
  - Nome della risorsa
  - Esempi
    - <http://xxx.myplace.com/www/index.html>
    - <http://xxx.myplace.com:80/cgi-bin/t.exe>

## Metodi HTTP (1)

- **GET**
  - Recupera un URL dal server
    - richiede una pagina
    - esegue un programma CGI
    - esegue un CGI con passandogli gli argomenti inviati insieme all'URL
- **POST**
  - Metodo preferito per elaborare le form
  - Esegue un programma CGI
  - Dati parametrizzati in sysin (dipende dal sistema operativo)
  - Più sicuro e privato

## Metodi HTTP (2)

- **PUT**
  - Usato per trasferire un file dal client al server
- **HEAD**
  - Richiede solo l'header dello stato degli URL
  - Usato negli schemi di miglioramento delle prestazioni
    - recupera l'URL solo se non è nella cache locale e se la data è più recente di quella della copia memorizzata nella cache

## Richieste

- Vengono inviate dal client al server
- Sono costituite dall'header HTTP e da un URL
  - header è nascosto dal browser
  - header contiene:
    - il tipo di contenuto/il tipo MIME
    - la lunghezza del contenuto
    - l'user agent – il browser che immette la richiesta
    - i tipi di contenuto che l'user agent può gestire

## Header delle richieste

- Precedono le richieste dei metodi HTTP
- Devono terminare con una riga vuota
- Campi degli header:
  - **From**
  - **Accept**
  - **Accept-Encoding**
  - **Accept Language**
  - **Referer**
  - **Authorization**
  - **Charge-To**
  - **If-Modified-Since**
  - **Pragma**

## **Il campo From**

- Indica l'indirizzo dell'utente che ha fatto la richiesta
  - Già visto nel protocollo SMTP
- Non deve necessariamente corrispondere al nome dell'host richiedente (potrebbe essere un proxy)
- Dovrebbe essere un indirizzo di posta elettronica valido

## **Il campo Accept**

- Elenco di schemi accettati dal client
  - Può contenere caratteri jolly (\*)
- Grammatica:
  - `<field> = Accept: <entry> * [,<entry>]`
  - `<entry> = <content type> * [<param>]`
  - `<param> = <attr> = <float>`
  - `<attr> = q / mxs / mxb`
  - `<float> = <ANSI-C floating point >`
- Esempi:
  - Accept: text/html
  - Accept: audio/basic q-1
- Se non viene trovato alcun campo Accept, si dà per scontato il testo normale

## **Il campo Accept-Encoding**

- È come Accept, ma l'elenco è un elenco di schemi di codifica accettabili
- Esempio:
  - Accept-Encoding: x-compress;x-zip

## **Il campo User-Agent**

- Prodotto software usato dal client
  - <field> = User-Agent: <product>
  - <product> = <word> [/<version>]
  - <version> = <word>
- Esempio:
  - User-Agent: IBM WebExplorer DLL /v960311

## **Il campo Referer**

- A vantaggio del server, il client elenca l'URL o il documento (o il tipo di documento) da cui è stato ottenuto l'URL nella richiesta
- Consente al server di creare backlink, log e tracce di collegamenti errati
- Esempio:
  - Referer: `http://www.w3.com/xxx.html`

## **Il campo Authorization**

- Per schemi di autenticazione e password
- Esempio:
  - Authorization: `user fred:mypassword`
  - Authorization: `kerberos kerberosparameters`

### **Il campo Charge-To**

- Informazioni di accounting
- Dipende dal sistema di accounting

### **Il campo Pragma**

- Ha lo stesso formato di Accept
- È rivolto ai server
- Dovrebbe essere passato attraverso i proxy, ma oggi viene usato dai proxy
- Il solo Pragma definito al momento è «no-cache»
  - I proxy dovrebbero ottenere il documento dal server originale piuttosto che dalla cache



### **Il campo If-Modified-Since**

- Viene usato con GET per eseguire un GET condizionale
- Se il documento richiesto non è stato modificato dalla data specificata, al client viene rimandato un header Modified 304 e non un documento
  - Il client poi può visualizzare la versione memorizzata nella sua cache

### **Pacchetti di risposta**

- Vengono inviati dal server al browser in risposta a un pacchetto di richiesta

## Header di stato

- “HTTP/1.0 sp code”
- Codici:
  - **1xx**: rimandato a un uso successivo
  - **2xx**: l'azione è stata ricevuta con successo, compresa ed accettata
  - **3xx**: per soddisfare la richiesta sono necessarie ulteriori azioni
  - **4xx**: la richiesta del client è sintatticamente scorretta
  - **5xx**: il server non può soddisfare la richiesta

## Header delle risposte

- Inviato dal server al browser lato client
- Header di stato
  - Entità
    - Content-Encoding:
    - Content-Length:
    - Content-Type:
    - Expires:
    - Last-Modified:
    - extension-header
- Corpo-contenuto (di solito html)

## Senza stato

- A causa della natura dei metodi Connect, Request, Response e Disconnect, HTTP è detto **“protocollo senza stato”**
- Nel protocollo HTTP da una pagina web alla successiva non c'è nulla che consenta a un programma web di mantenere lo “stato” del programma (come un programma del desktop)

